A General Implementation of Forward Propagation

Instead of hard-coding the computation for every single neuron, we can create a general-purpose function, `dense`, to compute the output of an entire layer.

The `dense` Function 🧑‍💻

The goal is to create a function that computes the activations for one layer.

- **Function definition:** `def dense(a_in, W, b)`
- **Inputs:**
  - `a_in`: The activation vector from the **previous** layer (for the first layer, this would be the input `x`).
  - `W`: The weight parameters for the **current** layer.
  - `b`: The bias parameters for the **current** layer.
- **Returns:**
  - `a_out`: The activation vector of the **current** layer.

Parameter Organization

To make this function work, we must organize our parameters into matrices and vectors.

- `W` **(Weight Matrix):** The weight vectors for each neuron in the layer are stacked as **columns** to form a matrix.
  - If Layer 1 has 3 neurons, `W1` would be a matrix where the 1st column is $\vec{w}_1^{[1]}$, the 2nd column is $\vec{w}_2^{[1]}$, and the 3rd column is $\vec{w}_3^{[1]}$.
- `b` **(Bias Vector):** The bias values for each neuron are stored in a 1D array.
  - `b1 = [b^{[1]}_1, b^{[1]}_2, b^{[1]}_3]`

Implementation Logic of the `dense` Function

The function iterates through each neuron (each column of `W`) and performs the standard logistic regression calculation.

```
# Pseudocode for the dense function
def dense(a_in, W, b):

    # Get the number of neurons (units) from the number of columns in W
    units = W.shape[1]

```

```
    # Initialize the output activation vector
    a_out = np.zeros(units)

    # Loop over each neuron
    for j in range(units):

        # Get the weight vector for the j-th neuron (the j-th column)
        w = W[:, j]

        # Calculate z = w_j . a_in + b_j
        z = np.dot(a_in, w) + b[j]

        # Calculate the activation for the j-th neuron
        a_out[j] = g(z)  # g is the sigmoid function

    return a_out
```

Building the Full Network

With this `dense` function, you can build a full neural network by "stringing together" the layers sequentially, passing the output of one layer as the input to the next.

```
# Example of a 4-layer network
def my_network(x, W1, b1, W2, b2, W3, b3, W4, b4):

    a1 = dense(x,  W1, b1)
    a2 = dense(a1, W2, b2)
    a3 = dense(a2, W3, b3)
    a4 = dense(a3, W4, b4)

    f_x = a4
    return f_x
```

Why Implement This Manually?

- Although in practice you will use libraries like TensorFlow, understanding what happens "under the hood" is critical.
- This knowledge is essential for **debugging** your models when they don't work, run slowly, or produce unexpected results.